FH-OÖ Hagenberg/HSD
SDP3, WS 2026
Übung 4

Name: T. Lechner, P. Ackermann | Aufwand in h: 12

Mat.Nr: S2510306017, S2510306001 | Punkte:

Übungsgruppe: 1 | korrigiert:

Beispiel 1 (24 Punkte) Bilderverwaltung: Erstellen Sie eine Software zur Erzeugung und Verwaltung von Bildern mit graphischen Objekten. Die Bilder sind in einer Bilderverwaltung abgelegt und können durch generieren einer SVG-Datei in einem entsprechenden Browser angezeigt werden.

Eine Methode `WriteSVG` soll das Bild als SVG-Datei (*scalable vector graphics*) mit dem gegebenen Namen abspeichern. Der Name der SVG-Datei wird bei der Konstruktion eines Bildes festgelegt. Ein Beispiel für eine solche SVG-Datei könnte z.B. folgendermaßen aussehen:

```
<?xml version="1.0"?>
<svg xmlns="http://www.w3.org/2000/svg">
<rect x="10" y="10" width="90" height="70" stroke="black" fill="none"/>
<circle cx="55" cy="45" r="35" stroke="black" fill="none"/>
</svg>
```

Die SVG-Datei beschreibt die Bestandteile der Grafik (im obigen Beispiel ein Rechteck und ein Kreis). Stroke (`stroke="black"`) gibt die Farbe an und `fill` die Füllung. Wird `fill="none"` weggelassen, so wird das Objekt mit der angegebenen Farbe gefüllt.

Es sollen gefüllte und nicht gefüllte Kreise und Rechtecke erzeugt werden, wobei ein Bild nur gefüllte oder nur nicht gefüllte Objekte haben kann.

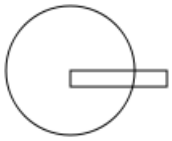


Bild1:test1.svg



Bild2: test2.svg

Die Bilderverwaltung und die Fabriken zur Objekterzeugung exisitieren nur einmal und werden beim

Beenden des Programmes automatisch freigegeben. Verwenden Sie zur Implementierung eine vorgestellte Methode aus der Vorlesung. Graphische Objekte werden mit einer entsprechenden Fabrik erzeugt und das Kopieren und Zuweisen soll verhindert werden!

Die Bilderverwaltung kann mit verschiedenen Fabriken arbeiten und so verschiedene Bilder erzeugen. Die Fabriken in der Bilderverwaltung sollen austauschbar sein!

Das Design soll so ausgelegt sein, dass in Zukunft nicht nur gefüllte und nicht gefüllte graphische Objekte erzeugt werden können, sondern auch z.B. schraffierte oder gepunktete graphische Objekte.

Für die SVG-Datei muss keine Grammatik angegeben werden!

Entwerfen Sie ein Klassendiagramm und verwenden Sie dazu entsprechende Design Pattern. Implementieren Sie das vollständige Klassendiagramm und schreiben sie einen Testtreiber. Verfassen Sie weiters eine Systemdokumentation (Funktionalität, Klassendiagramm, Schnittstellen der beteiligten Klassen, etc.)! Geben Sie die vom Testtreiber erzeugten SVG-Dateien sowohl textuell als auch graphisch ab.

Allgemeine Hinweise: Legen Sie bei der Erstellung Ihrer Übung großen Wert auf eine **saubere Strukturierung** und auf eine **sorgfältige Ausarbeitung!** Dokumentieren Sie alle Schnittstellen und versehen Sie Ihre Algorithmen an entscheidenden Stellen ausführlich mit Kommentaren! Testen Sie ihre Implementierungen ausführlich! Geben Sie den **Testoutput** mit ab!



Systemdokumentation Projekt Bilderverwaltung

Version 1.0

T. Lechner, P. Ackermann

Hagenberg, 10. November 2026

Inhaltsverzeichnis

1 Organisatorisches **5**
1.1 Team 5
1.2 Aufteilung der Verantwortlichkeitsbereiche 5
1.3 Aufwand 6

2 Anforderungsdefinition (Systemspezifikation) **6**

3 Systementwurf **7**
3.1 Klassendiagramm 7
3.2 Designentscheidungen 7

4 Dokumentation der Komponenten (Klassen) **9**

5 Testprotokollierung **9**

6 Quellcode **11**
6.1 Object 11
6.2 SingletonBase 12
6.3 GraphicObject 13
6.3.1 Rectangles 14
6.3.2 Circles 18
6.4 GraphicObjectFactory 21
6.4.1 FilledObjectFactory 22
6.4.2 FramedObjectFactory 23
6.5 Image 25
6.6 ImageManager 27
6.7 Testtreiber 29

1 Organisatorisches

1.1 Team

- T. Lechner, Matr.-Nr.: S2510306017, E-Mail: s2510306017@students.fh-hagenberg.at
- P. Ackermann, Matr.-Nr.: S2510306001, E-Mail: s2510306001@students.fh-hagenberg.at

1.2 Aufteilung der Verantwortlichkeitsbereiche

- T. Lechner
  - Klassendiagramm
  - Designentscheidungen
  - Implementierung und Unit-Test:
    - ImageManager
    - Image
    - GraphicObject
    - Rectangle
    - FramedRectangle
    - FilledRectangle
- P. Ackermann
  - Klassendiagramm
  - Anforderungsdefinition
  - Implementierung und Unit-Test:

* Object
* SingletonBase
* GraphicObjectFactory
* FramedObjectFactory
* FilledObjectFactory
* Circle
* FramedCircle
* FilledCircle

1.3 Aufwand

- T. Lechner: geschätzt 7 Ph / tatsächlich 6 Ph
- P. Ackermann: geschätzt 7 Ph / tatsächlich 6 Ph

2 Anforderungsdefinition (Systemspezifikation)

Es soll eine Software zur Verwaltung von *Graphic Objects* in Bildern erstellt werden. Diese *Bilder* sollen im Format SVG mit einem zugehörigen Namen gespeichert werden. Ein *Bild* kann mehrfach die *Graphic Objects* Rechteck und Kreis enthalten, wobei diese Objekte durch verschiedene Eigenschaften wie die Position oder den spezifischen Größen der Dimensionierung beschrieben werden. Des Weiteren soll jedes *Graphic Object* in einem gewissen 'Stil' dargestellt werden können. Es sollen entweder gefüllte oder nicht gefüllte Rechtecke oder Kreise erstellt werden können. Allerdings sollte ein *Bild* nur *Graphic Objects* von **einem** 'Stil' enthalten können. Somit soll die Software auch für spätere Erweiterung von Stilen implementiert werden.

3 Systementwurf

3.1 Klassendiagramm

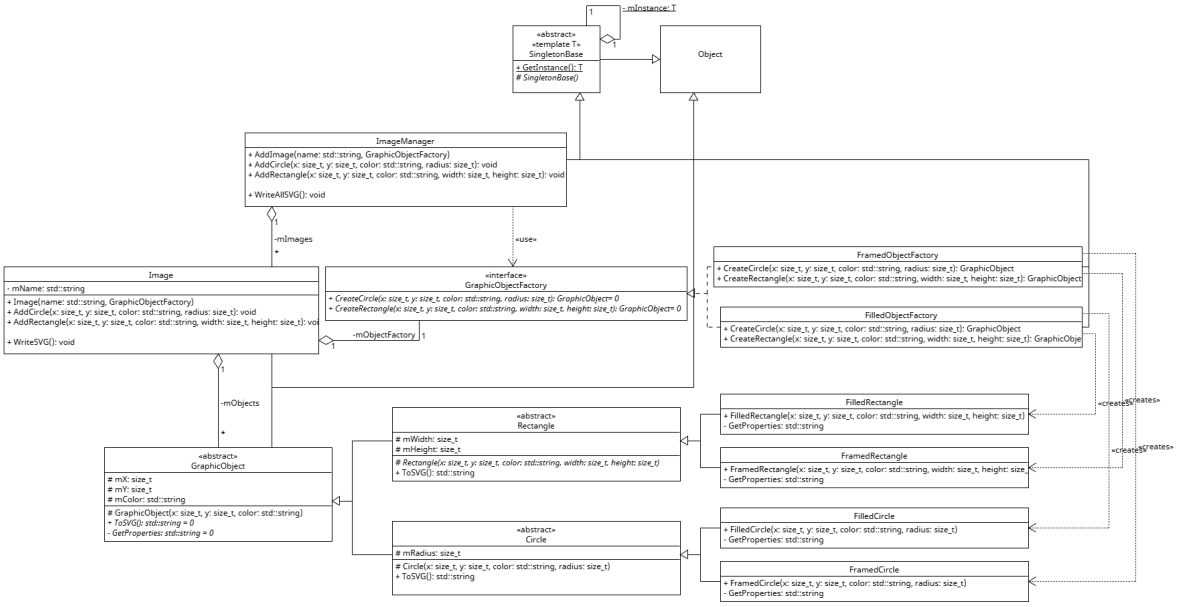


Abbildung 1: Klassendiagramm Bilderverwaltung

3.2 Designentscheidungen

Object wird wieder als Basisklasse aller Basisklassen verwendet, damit der virtuelle Destruktor überall vorhanden ist.

Die **ImageManager** Klasse ist die Hauptklasse der Anwendung, die dann vom Client verwendet wird. Man kann ein neues **Image** mit einer gewissen **GraphicObjectFactory** anlegen und dann immer auf dem neu angelegten **Image** neue **Circle**s und **Rectangle**s hinzufügen, bis ein neues **Image** hinzugefügt wird.

Circle und **Rectangle** erben von der abstrakten **GraphicObject** Klasse und sind aber auch selbst abstrakt, da sie noch jeweils von **FilledCircle**/**FramedCircle** bzw. **FilledRectangle**/**FramedRectangle** abgeleitet werden.

GraphicObject hat als Member nur die Position (X,Y) und die Farbe, **Circle** zusätzlich noch einen Radius und **Rectangle** zusätzlich Höhe und Breite.

Die Grafikobjekte werden immer von einer **GraphicObjectFactory** (abgeleitete Klassen **FilledObjectFactory** und **FramedObjectFactory**) konstruiert. **FilledObjectFactory** konstruiert immer die **Filled**-Varianten und **FramedObjectFactory** die **Framed**-Varianten.

Beim Konstruieren eines **Images** wird eine solche Fabrik mitgegeben. Mit dieser werden in *AddCircle* und *AddRectangle* neue Grafikobjekte erstellt, die intern in einer Liste gespeichert werden → **Abstract Factory Pattern**.

Die Fabriken und **ImageManager** sollen nur ein Mal gleichzeitig existieren können, dafür wird das **Singleton Pattern** verwendet. Genauer gesagt wird von der abstrakten **SingletonBase** Template-Klasse geerbt. Diese erbt von **Object**, da sie auch einen virtuellen Destruktor benötigt.

Zur Umwandlung der Grafikobjekt-Daten ins SVG-Format, wird die *ToSVG* Methode eines **GraphicObject**s verwendet, die pure virtual ist und von **Circle** und **Rectangle** implementiert wird. Dabei werden die Daten als XML-SVG-String zurückgegeben. Da die Kindklassen von **Rectangle** und **Circle** eventuell zusätzliche Eigenschaften zum XML hinzufügen müssen (z.B. fill="none"), gibt es noch eine weitere pure virtual Methode *GetProperties*. Diese wird in den untersten Klassen implementiert und von den abstrakten **Circle** und **Rectangle** in deren *ToSVG* Methode aufgerufen. Dadurch kann die Erstellung des XMLs zusätzlich angepasst werden → **Template Method Pattern**.

In *WriteSVG* generiert **Image** so die XML-SVG-Strings für jedes gespeicherte **GraphicObject** und schreibt diese Strings zusätzlich mit einem XML-Header und XML-Footer in eine Datei. Der Name der Datei entspricht dem Namen des **Image**s (Member *mName*) mit .svg Endung. **ImageManager** ruft in *WriteAllSVG* für jedes gespeicherte **Image** die *WriteSVG* Methode auf.

4 Dokumentation der Komponenten (Klassen)

Die HTML-Startdatei befindet sich im Verzeichnis ./../doxy/html/index.html

5 Testprotokollierung

Die Tests wurden mittels Google Test Suite direkt in Visual Studio durchgeführt.
Die generierten SVG-Dateien befinden sich im Ordner **Beispiel01**. *Img2.svg* ist leer.

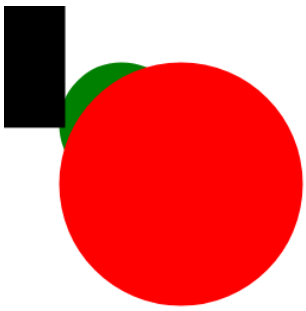


Abbildung 2: TestImage.svg

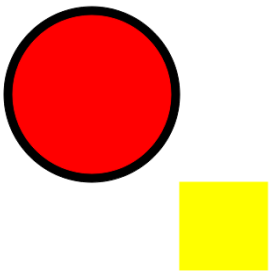


Abbildung 3: Img3.svg

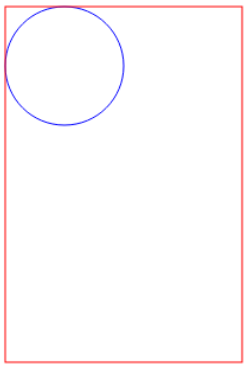


Abbildung 4: Img1.svg

Listing 1: Systemdoku/Tests

```
Visual Leak Detector read settings from: C:\Program Files (x86)\
    Visual Leak Detector\vld.ini
Visual Leak Detector Version 2.5.1 installed.
Running main() from D:\a\_work\1\s\googletest\googletest\src\
    gtest_main.cc
[==========] Running 10 tests from 8 test cases.
[----------] Global test environment set-up.
[----------] 1 test from FramedRectangle
[ RUN      ] FramedRectangle.Test
[       OK ] FramedRectangle.Test (0 ms)
[----------] 1 test from FramedRectangle (1 ms total)

[----------] 1 test from FilledRectangle
[ RUN      ] FilledRectangle.Test
[       OK ] FilledRectangle.Test (0 ms)
[----------] 1 test from FilledRectangle (1 ms total)

[----------] 1 test from FramedCircle
[ RUN      ] FramedCircle.Test
[       OK ] FramedCircle.Test (0 ms)
[----------] 1 test from FramedCircle (1 ms total)

[----------] 1 test from FilledCircle
[ RUN      ] FilledCircle.Test
[       OK ] FilledCircle.Test (0 ms)
[----------] 1 test from FilledCircle (1 ms total)

[----------] 1 test from FramedObjectFactory
```

```
[ RUN      ] FramedObjectFactory.Test
[       OK ] FramedObjectFactory.Test (0 ms)
[----------] 1 test from FramedObjectFactory (1 ms total)

[----------] 1 test from FilledObjectFactory
[ RUN      ] FilledObjectFactory.Test
[       OK ] FilledObjectFactory.Test (1 ms)
[----------] 1 test from FilledObjectFactory (1 ms total)

[----------] 2 tests from Image
[ RUN      ] Image.Test
[       OK ] Image.Test (4 ms)
[ RUN      ] Image.InvldName
[       OK ] Image.InvldName (0 ms)
[----------] 2 tests from Image (6 ms total)

[----------] 2 tests from ImageManager
[ RUN      ] ImageManager.Test
[       OK ] ImageManager.Test (9 ms)
[ RUN      ] ImageManager.InvldName
[       OK ] ImageManager.InvldName (2 ms)
[----------] 2 tests from ImageManager (12 ms total)

[----------] Global test environment tear-down
[==========] 10 tests from 8 test cases ran. (31 ms total)
[  PASSED  ] 10 tests.
No memory leaks detected.
Visual Leak Detector is now exiting.
```

6 Quellcode

6.1 Object

Listing 2: "Beispiel01/Object.h"

```
1  #ifndef OBJECT_H
2  #define OBJECT_H
3
4  /// @author Philip
5  /// @date 5.11.2026
6  /// @brief Base class of all base classes.
7  ///
8  /// Is the common root class of all classes. If there is no other
       base class,
9  /// this class is to be selected as the base class.
10 class Object
11 {
12 public:
```

```
13      /// @brief Virtual default destructor -> once virtual, always
            virtual.
14      ///
15      /// It is not necessary to declare another virtual destructor
16      /// in derived classes.
17      virtual ~Object() = default;
18
19  protected:
20      /// @brief Protected default constructor that makes the class
            abstract.
21      Object() = default;
22  };
23  #endif
```

6.2 SingletonBase

Listing 3: "Beispiel01/SingletonBase.h"

```
1   #ifndef SINGLETONBASE_H
2   #define SINGLETONBASE_H
3
4   #include "Object.h"
5
6   #include <memory>
7
8
9   /// @author Philip
10  /// @date 5.11.2026
11  /// @brief SingletonBase class to derive other singleton classes.
12  ///
13  /// This class implements the shared methods of each derived
        Singleton class
14  template <typename T> class SingletonBase : public Object
15  {
16  public:
17
18      /// @brief constructes the object of the class
19      /// @returns the constructed object of the class
20      static T& GetInstance()
21      {
22          if (mInstance == nullptr)
23          {
24              // add Object only if it does not already exist
25              mInstance = std::unique_ptr<T>{ new T{} };
26          }
27          return *mInstance;
28      }
29
30  protected:
31
32      /// @brief CTor
```

```
33      SingletonBase() = default;
34
35
36  private:
37      /// @brief CCTor
38      ///
39      /// must be deleted or private (or both), because only one
            instance of class must exist
40      SingletonBase(SingletonBase const& other) = delete;
41
42      /// @brief Assignment operator
43      ///
44      /// deleted or private (or both), because no other instance of
             class can exist that could be assigned
45      SingletonBase& operator = (SingletonBase const& other) =
            delete;
46
47      /// @brief Uniqueptr on the instance
48      static std::unique_ptr<T> mInstance;
49
50  };
51  // init static member
52  template <typename T> std::unique_ptr<T> SingletonBase<T>::
        mInstance = nullptr;
53
54  #endif
```

6.3 GraphicObject

Listing 4: Beispiel01/GraphicObject.h

```
1   #ifndef GRAPHICOBJECT_H
2   #define GRAPHICOBJECT_H
3
4   #include "Object.h"
5
6   #include <string>
7   #include <memory>
8
9
10  /// @author Thomas Lechner
11  /// @date 05.11.2026
12  /// @brief Base class of all graphic objects.
13  ///
14  /// Stores common information like position and color of the
        graphic object.
15  class GraphicObject : public Object
16  {
17  public:
18      /// @brief std::unique_ptr of this class.
19      using UPtr = std::unique_ptr<GraphicObject>;
```

```
20
21      /// @brief CTor.
22      /// @param[in] x The X-coordinate of position.
23      /// @param[in] y The Y-coordinate of position.
24      /// @param[in] color Color of the graphic object (e.g. black,
            green,...).
25      GraphicObject(size_t const x, size_t const y, std::string
            const& color);
26
27      /// @brief Creates a SVG-XML-Element of the graphic object.
28      /// @returns The SVG-XML-Element as a string.
29      virtual std::string ToSVG() const = 0;
30
31  protected:
32      /// @brief Gets additional properties for the SVG-XML-Element.
33      /// @returns XML-properties as a string.
34      ///
35      /// This is used as a template method in ToSVG for SVG
            creation.
36      virtual std::string GetProperties() const = 0;
37
38      /// @ X-coordinate of position.
39      size_t const mX;
40      /// @ Y-coordinate of position.
41      size_t const mY;
42
43      /// @brief Color of the graphic object (e.g. black, green,...)
            .
44      std::string const mColor;
45  };
46
47  #endif
```

Listing 5: Beispiel01/GraphicObject.cpp

```
1  #include "GraphicObject.h"
2
3  GraphicObject::GraphicObject(size_t const x, size_t const y, std::
       string const& color) :
4      mX{ x },
5      mY{ y },
6      mColor{ color }
7  {
8  }
```

6.3.1 Rectangles

Listing 6: Beispiel01/Rectangle.h

```
1  #ifndef RECTANGLE_H
```

```
2  #define RECTANGLE_H
3
4  #include "GraphicObject.h"
5
6
7  /// @author Thomas Lechner
8  /// @date 07.11.2026
9  /// @brief Abstract base class of all rectangles.
10 ///
11 /// Stores common information like height and width of the
       rectangle.
12 class Rectangle : public GraphicObject
13 {
14 public:
15     /// @brief std::unique_ptr of this class.
16     using UPtr = std::unique_ptr<Rectangle>;
17
18     /// @brief CTor.
19     /// @param[in] x The X-coordinate of position.
20     /// @param[in] y The Y-coordinate of position.
21     /// @param[in] color Color of the rectangle (e.g. black, green
           ,...).
22     /// @param[in] width The width of the rectangle.
23     /// @param[in] height The height of the rectangle.
24     Rectangle(size_t const x, size_t const y, std::string const&
           color, size_t const width, size_t const height);
25
26     /// @brief Creates a SVG-XML-Element of the rectangle.
27     /// @returns The SVG-XML-Element as a string.
28     std::string ToSVG() const override;
29
30 protected:
31     /// @brief The width of the rectangle.
32     size_t const mWidth;
33     /// @brief The height of the rectangle.
34     size_t const mHeight;
35 };
36
37 #endif
```

Listing 7: Beispiel01/Rectangle.cpp

```
1  #include "Rectangle.h"
2
3  #include <format>
4
5  Rectangle::Rectangle(size_t const x, size_t const y, std::string
       const& color, size_t const width, size_t const height) :
6      GraphicObject{ x, y, color },
7      mWidth{ width },
8      mHeight{ height }
9  {
10 }
11
12 std::string Rectangle::ToSVG() const
```

```
13 {
14     // Example: <rect x="10" y="10" width="90" height="70" stroke
           ="black" fill="none"/>
15     return std::format("<rect x=\"{}\" y=\"{}\" width=\"{}\"
           height=\"{}\" stroke=\"{}\" {}/>", mX, mY, mWidth, mHeight
           , mColor, GetProperties());
16 }
```

Listing 8: Beispiel01/FilledRectangle.h

```
1  #ifndef FILLEDRECTANGLE_H
2  #define FILLEDRECTANGLE_H
3
4  #include "Rectangle.h"
5
6
7  /// @author Thomas Lechner
8  /// @date 07.11.2026
9  /// @brief Class of a filled rectangle.
10 ///
11 /// Represents a rectangle with its area drawn as filled.
12 class FilledRectangle : public Rectangle
13 {
14 public:
15     /// @brief std::unique_ptr of this class.
16     using UPtr = std::unique_ptr<FilledRectangle>;
17
18     /// @brief CTor.
19     /// @param[in] x The X-coordinate of position.
20     /// @param[in] y The Y-coordinate of position.
21     /// @param[in] color Color of the rectangle (e.g. black, green
           ,...).
22     /// @param[in] width The width of the rectangle.
23     /// @param[in] height The height of the rectangle.
24     FilledRectangle(size_t const x, size_t const y, std::string
           const& color, size_t const width, size_t const height);
25
26 protected:
27     /// @brief Gets additional properties for the SVG-XML-Element.
28     /// @returns XML-properties as a string.
29     ///
30     /// This is used as a template method in ToSVG for SVG
           creation.
31     std::string GetProperties() const override;
32 };
33
34 #endif
```

Listing 9: Beispiel01/FilledRectangle.cpp

```
1 #include "FilledRectangle.h"
2
3 #include <format>
```

```
4
5  FilledRectangle::FilledRectangle(size_t const x, size_t const y,
       std::string const& color, size_t const width, size_t const
       height) :
6      Rectangle{ x, y, color, width, height }
7  {
8  }
9
10 std::string FilledRectangle::GetProperties() const
11 {
12     return std::format("fill=\"{}\"", mColor);
13 }
```

Listing 10: Beispiel01/FramedRectangle.h

```
1  #ifndef FRAMEDRECTANGLE_H
2  #define FRAMEDRECTANGLE_H
3
4  #include "Rectangle.h"
5
6
7  /// @author Thomas Lechner
8  /// @date 07.11.2026
9  /// @brief Class of a framed rectangle.
10 ///
11 /// Represents a rectangle drawn as a frame.
12 class FramedRectangle : public Rectangle
13 {
14 public:
15     /// @brief std::unique_ptr of this class.
16     using UPtr = std::unique_ptr<FramedRectangle>;
17
18     /// @brief CTor.
19     /// @param[in] x The X-coordinate of position.
20     /// @param[in] y The Y-coordinate of position.
21     /// @param[in] color Color of the rectangle (e.g. black, green
           ,...).
22     /// @param[in] width The width of the rectangle.
23     /// @param[in] height The height of the rectangle.
24     FramedRectangle(size_t const x, size_t const y, std::string
           const& color, size_t const width, size_t const height);
25
26 protected:
27     /// @brief Gets additional properties for the SVG-XML-Element.
28     /// @returns XML-properties as a string.
29     ///
30     /// This is used as a template method in ToSVG for SVG
           creation.
31     std::string GetProperties() const override;
32 };
33
34 #endif
```

Listing 11: Beispiel01/FramedRectangle.cpp

```cpp
#include "FramedRectangle.h"

FramedRectangle::FramedRectangle(size_t const x, size_t const y,
    std::string const& color, size_t const width, size_t const
    height) :
    Rectangle{ x, y, color, width, height }
{
}

std::string FramedRectangle::GetProperties() const
{
    // Makes the rectangle fill transparent.
    return "fill=\"none\"";
}
```

6.3.2 Circles

Listing 12: Beispiel01/Circle.h

```cpp
#ifndef CIRCLE_H
#define CIRCLE_H

#include "GraphicObject.h"


/// @author Philip Ackermann
/// @date 09.11.2026
/// @brief Abstract base class of all circles.
///
/// Stores common information like radius of the circle.
class Circle : public GraphicObject
{
public:
    /// @brief std::unique_ptr of this class.
    using UPtr = std::unique_ptr<Circle>;

    /// @brief CTor.
    /// @param[in] x The X-coordinate of position.
    /// @param[in] y The Y-coordinate of position.
    /// @param[in] color Color of the circle (e.g. black, green
        ,...).
    /// @param[in] width The width of the circle.
    Circle(size_t const x, size_t const y, std::string const&
        color, size_t const radius);

    /// @brief Creates a SVG-XML-Element of the circle.
    /// @returns The SVG-XML-Element as a string.
    std::string ToSVG() const override;

```

```
29 protected:
30     /// @brief The radius of the circle.
31     size_t const mRadius;
32
33 };
34
35 #endif
```

Listing 13: Beispiel01/Circle.cpp

```
1  #include "Circle.h"
2
3  #include <format>
4
5  Circle::Circle(size_t const x, size_t const y, std::string const&
       color, size_t const radius) :
6      GraphicObject{ x, y, color },
7      mRadius{radius}
8  {
9  }
10
11 std::string Circle::ToSVG() const
12 {
13     // Example: <circle cx="55" cy="45" r="35" stroke="black" fill
           ="none"/>
14     return std::format("<circle cx=\"{}\" cy=\"{}\" r=\"{}\"
           stroke=\"{}\" {}/>", mX, mY, mRadius, mColor,
           GetProperties());
15 }
```

Listing 14: Beispiel01/FilledCircle.h

```
1  #ifndef FILLEDCIRCLE_H
2  #define FILLEDCIRCLE_H
3
4  #include "Circle.h"
5
6
7  /// @author Philip Ackermann
8  /// @date 09.11.2026
9  /// @brief Class of a filled circle.
10 ///
11 /// Represents a circle with its area drawn as filled.
12 class FilledCircle : public Circle
13 {
14 public:
15     /// @brief std::unique_ptr of this class.
16     using UPtr = std::unique_ptr<FilledCircle>;
17
18     /// @brief CTor.
19     /// @param[in] x The X-coordinate of position.
20     /// @param[in] y The Y-coordinate of position.
```

```
21      /// @param[in] color Color of the circle (e.g. black, green
            ,...).
22      /// @param[in] radius The radius of the circle.
23      FilledCircle(size_t const x, size_t const y, std::string const
            & color, size_t const radius);
24
25  protected:
26      /// @brief Gets additional properties for the SVG-XML-Element.
27      /// @returns XML-properties as a string.
28      ///
29      /// This is used as a template method in ToSVG for SVG
            creation.
30      std::string GetProperties() const override;
31  };
32
33  #endif
```

Listing 15: Beispiel01/FilledCircle.cpp

```
1   #include "FilledCircle.h"
2
3   #include <format>
4
5   FilledCircle::FilledCircle(size_t const x, size_t const y, std::
        string const& color, size_t const radius) :
6       Circle{x,y,color,radius}
7   {
8   }
9
10  std::string FilledCircle::GetProperties() const
11  {
12      return std::format("fill=\"{}\"", mColor);
13  }
```

Listing 16: Beispiel01/FramedCircle.h

```
1   #ifndef FRAMEDCIRCLE_H
2   #define FRAMEDCIRCLE_H
3
4   #include "Circle.h"
5
6
7   /// @author Philip Ackermann
8   /// @date 09.11.2026
9   /// @brief Class of a framed circle.
10  ///
11  /// Represents a circle drawn as frame.
12  class FramedCircle : public Circle
13  {
14  public:
15      /// @brief std::unique_ptr of this class.
16      using UPtr = std::unique_ptr<FramedCircle>;
17
```

```
18      /// @brief CTor.
19      /// @param[in] x The X-coordinate of position.
20      /// @param[in] y The Y-coordinate of position.
21      /// @param[in] color Color of the circle (e.g. black, green
            ,...).
22      /// @param[in] radius The radius of the circle.
23      FramedCircle(size_t const x, size_t const y, std::string const
            & color, size_t const radius);
24
25  protected:
26      /// @brief Gets additional properties for the SVG-XML-Element.
27      /// @returns XML-properties as a string.
28      ///
29      /// This is used as a template method in ToSVG for SVG
            creation.
30      std::string GetProperties() const override;
31  };
32
33  #endif
```

Listing 17: Beispiel01/FramedCircle.cpp

```
1   #include "FramedCircle.h"
2
3   FramedCircle::FramedCircle(size_t const x, size_t const y, std::
        string const& color, size_t const radius) :
4       Circle{ x,y,color,radius }
5   {
6   }
7
8   std::string FramedCircle::GetProperties() const
9   {
10      // Makes the rectangle fill transparent.
11      return "fill=\"none\"";
12  }
```

6.4 GraphicObjectFactory

Listing 18: "Beispiel01/GraphicObjectFactory.h"

```
1   #ifndef GRAPHICOBJECTFACTORY_H
2   #define GRAPHICOBJECTFACTORY_H
3
4   #include <string>
5   #include <memory>
6
7   #include "GraphicObject.h"
8
9
10  /// @author Philip
```

```
11 /// @date 5.11.2026
12 /// @brief Interface class for diffrent graphic object factories
13 ///
14 /// Circles and rectangle in diffrent styles can be created
15 class GraphicObjectFactory
16 {
17 public:
18     /// @brief Creates a circle
19     /// @param[in] x x-pos of the circle
20     /// @param[in] y y-pos of the circle
21     /// @param[in] color color of the circle
22     /// @param[in] radius radius of the circle
23     virtual GraphicObject::UPtr CreateCircle(size_t const x,
           size_t const y, std::string const& color, size_t const
           radius) const = 0;
24
25
26     /// @brief Creates a rectangle
27     /// @param[in] x x-pos of the rectangle
28     /// @param[in] y y-pos of the rectangle
29     /// @param[in] color color of the rectangle
30     /// @param[in] width width of the rectangle
31     /// @param[in] height height of the rectangle
32     virtual GraphicObject::UPtr CreateRectangle(size_t const x,
           size_t const y, std::string const& color, size_t const
           width, size_t const height) const = 0;
33
34     /// @brief Virtual DTor.
35     virtual ~GraphicObjectFactory() = default;
36 };
37
38 #endif
```

6.4.1 FilledObjectFactory

Listing 19: Beispiel01/FilledObjectFactory.h

```
1  #ifndef FILLEDOBJECTFACTORY_H
2  #define FILLEDOBJECTFACTORY_H
3
4  #include "GraphicObjectFactory.h"
5  #include "SingletonBase.h"
6
7
8  /// @author Philip
9  /// @date 8.11.2026
10 /// @brief Factory to create filled circles and rectangles
11 class FilledObjectFactory : public GraphicObjectFactory, public
       SingletonBase<FilledObjectFactory>
12 {
13     friend class SingletonBase<FilledObjectFactory>;
14 public:
```

```
15      /// @brief Creates a filled circle
16      /// @param[in] x x-pos of the circle
17      /// @param[in] y y-pos of the circle
18      /// @param[in] color color of the circle
19      /// @param[in] radius radius of the circle
20      GraphicObject::UPtr CreateCircle(size_t const x, size_t const
            y, std::string const& color, size_t const radius) const
            override;
21
22
23      /// @brief Creates a filled rectangle
24      /// @param[in] x x-pos of the rectangle
25      /// @param[in] y y-pos of the rectangle
26      /// @param[in] color color of the rectangle
27      /// @param[in] width width of the rectangle
28      /// @param[in] height height of the rectangle
29      GraphicObject::UPtr CreateRectangle(size_t const x, size_t
            const y, std::string const& color, size_t const width,
            size_t const height) const override;
30  protected:
31      /// @brief Default CTor.
32      FilledObjectFactory() = default;
33  };
34
35  #endif
```

Listing 20: Beispiel01/FilledObjectFactory.cpp

```
1   #include "FilledObjectFactory.h"
2   #include "FilledRectangle.h"
3   #include "FilledCircle.h"
4
5   GraphicObject::UPtr FilledObjectFactory::CreateCircle(size_t const
        x, size_t const y, std::string const& color, size_t const
        radius) const
6   {
7       return std::make_unique<FilledCircle>(x, y, color,radius);
8   }
9
10  GraphicObject::UPtr FilledObjectFactory::CreateRectangle(size_t
        const x, size_t const y, std::string const& color, size_t
        const width, size_t const height) const
11  {
12      return std::make_unique<FilledRectangle>(x, y, color, width,
            height);
13  }
```

6.4.2 FramedObjectFactory

Listing 21: Beispiel01/FramedObjectFactory.h

```
#ifndef FRAMEDOBJECTFACTORY_H
#define FRAMEDOBJECTFACTORY_H

#include "GraphicObjectFactory.h"
#include "SingletonBase.h"


/// @author Philip
/// @date 8.11.2026
/// @brief Factory to create framed circles and rectangles
class FramedObjectFactory : public GraphicObjectFactory, public
    SingletonBase<FramedObjectFactory>
{
    friend class SingletonBase<FramedObjectFactory>;
public:
    /// @brief Creates a framed circle
    /// @param[in] x x-pos of the circle
    /// @param[in] y y-pos of the circle
    /// @param[in] color color of the circle
    /// @param[in] radius radius of the circle
    GraphicObject::UPtr CreateCircle(size_t const x, size_t const
        y, std::string const& color, size_t const radius) const
        override;


    /// @brief Creates a framed rectangle
    /// @param[in] x x-pos of the rectangle
    /// @param[in] y y-pos of the rectangle
    /// @param[in] color color of the rectangle
    /// @param[in] width width of the rectangle
    /// @param[in] height height of the rectangle
    GraphicObject::UPtr CreateRectangle(size_t const x, size_t
        const y, std::string const& color, size_t const width,
        size_t const height) const override;

protected:
    /// @brief Default CTor.
    FramedObjectFactory() = default;
};

#endif
```

Listing 22: Beispiel01/FramedObjectFactory.cpp

```
#include "FramedObjectFactory.h"
#include "FramedRectangle.h"
#include "FramedCircle.h"

GraphicObject::UPtr FramedObjectFactory::CreateCircle(size_t const
     x, size_t const y, std::string const& color, size_t const
    radius) const
{
    return std::make_unique<FramedCircle>(x, y, color, radius);
}
```

```
9
10 GraphicObject::UPtr FramedObjectFactory::CreateRectangle(size_t
       const x, size_t const y, std::string const& color, size_t
       const width, size_t const height) const
11 {
12     return std::make_unique<FramedRectangle>(x, y, color, width,
           height);
13 }
```

6.5 Image

Listing 23: Beispiel01/Image.h

```
1  #ifndef IMAGE_H
2  #define IMAGE_H
3
4  #include "Object.h"
5  #include "GraphicObject.h"
6  #include "GraphicObjectFactory.h"
7
8  #include <string>
9  #include <vector>
10
11
12 /// @author Thomas Lechner
13 /// @date 07.11.2026
14 /// @brief SVG-Image consisting of multiple GraphicObject.
15 ///
16 /// You can add new Objects and write the SVG-Image to a file.
17 class Image : public Object
18 {
19 public:
20     /// @brief std::unique_ptr of this class.
21     using UPtr = std::unique_ptr<Image>;
22
23     /// @brief CTor.
24     /// @param[in] name The name of the image -> will be the
           filename.
25     /// @param[in] factory The GraphicObjectFactory used to
           construct new GraphicObject.
26     Image(std::string const& name, GraphicObjectFactory& factory);
27
28
29     /// @brief Adds a new Rectangle to the Image.
30     /// @param[in] x The X-coordinate of position.
31     /// @param[in] y The Y-coordinate of position.
32     /// @param[in] color Color of the rectangle (e.g. black, green
           ,...).
33     /// @param[in] width The width of the rectangle.
34     /// @param[in] height The height of the rectangle.
```

```
35      ///
36      /// The Rectangle is constructed by mObjectFactory.
37      void AddRectangle(size_t const x, size_t const y, std::string
            const& color, size_t const width, size_t const height);
38
39
40      /// @brief Adds a new Circle to the Image.
41      /// @param[in] x The X-coordinate of position.
42      /// @param[in] y The Y-coordinate of position.
43      /// @param[in] color Color of the circle (e.g. black, green
            ,...).
44      /// @param[in] radius The radius of the circle.
45      ///
46      /// The Circle is constructed by mObjectFactory.
47      void AddCircle(size_t const x, size_t const y, std::string
            const& color, size_t const radius);
48
49      /// @brief Writes the Image to a file in SVG-Format.
50      ///
51      /// The name of the created file is <mName>.svg.
52      void WriteToSVG() const;
53
54  private:
55      /// @brief The name of the image -> will be the filename.
56      std::string const mName;
57      /// @brief The GraphicObjectFactory used to construct new
            GraphicObject.
58      GraphicObjectFactory const& mObjectFactory;
59
60      /// @brief List of all stored GraphicObject.
61      std::vector<GraphicObject::UPtr> mObjects;
62  };
63
64  #endif
```

Listing 24: Beispiel01/Image.cpp

```
1   #include "Image.h"
2
3   #include <format>
4   #include <fstream>
5
6   static auto const cErrFileOpen = "Error opening file";
7   static auto const cErrFileWrite = "Error writing file";
8   static auto const cErrFileClose = "Error closing file";
9   static auto const cErrName = "Invalid name";
10
11  Image::Image(std::string const& name, GraphicObjectFactory&
        factory) :
12      mName{ name },
13      mObjectFactory{ factory }
14  {
15      if (name.empty()) throw std::invalid_argument(cErrName);
16  }
17
```

```
18 void Image::AddRectangle(size_t const x, size_t const y, std::
       string const& color, size_t const width, size_t const height)
19 {
20     mObjects.emplace_back(mObjectFactory.CreateRectangle(x, y,
           color, width, height));
21 }
22
23 void Image::AddCircle(size_t const x, size_t const y, std::string
       const& color, size_t const radius)
24 {
25     mObjects.emplace_back(mObjectFactory.CreateCircle(x, y, color,
            radius));
26 }
27
28 void Image::WriteToSVG() const
29 {
30     std::ofstream file{ std::format("{}.svg", mName) };
31     if (!file.is_open()) throw std::runtime_error(cErrFileOpen);
32
33     file << "<?xml version=\"1.0\"?>\n<svg xmlns=\"http://www.w3.
           org/2000/svg\">\n";
34
35     for (auto const& obj : mObjects)
36     {
37         if (obj == nullptr) continue; // Cannot happen in the
               current implementation -> just for safety.
38         file << obj->ToSVG() << std::endl;
39     }
40
41     file << "</svg>";
42
43     if (!file.good()) throw std::runtime_error(cErrFileWrite);
44
45     file.close();
46
47     if (file.fail()) throw std::runtime_error(cErrFileClose);
48 }
```

6.6 ImageManager

Listing 25: Beispiel01/ImageManager.h

```
1 #ifndef IMAGEMANAGER_H
2 #define IMAGEMANAGER_H
3
4 #include "SingletonBase.h"
5 #include "Image.h"
6
7 #include <vector>
8
```

```
9
10 /// @author Thomas Lechner
11 /// @date 07.11.2026
12 /// @brief Manages multiple Image objects.
13 ///
14 /// Provides operations on Image objects.
15 class ImageManager : public SingletonBase<ImageManager>
16 {
17     friend class SingletonBase<ImageManager>;
18 public:
19     /// @brief Add a new Image to the ImageManager.
20     /// @param[in] name The name of the image -> will be the
        filename.
21     /// @param[in] factory The GraphicObjectFactory used to
        construct new GraphicObject.
22     ///
23     /// The Image is selected until the next call to this function
        . During this time it is possible to add GraphicObjects by
         using the Add methods.
24     void AddImage(std::string const& name, GraphicObjectFactory&
        factory);
25
26     /// @brief Adds a new Rectangle to the currently selected
        Image.
27     /// @param[in] x The X-coordinate of position.
28     /// @param[in] y The Y-coordinate of position.
29     /// @param[in] color Color of the rectangle (e.g. black, green
        ,...).
30     /// @param[in] width The width of the rectangle.
31     /// @param[in] height The height of the rectangle.
32     ///
33     /// The Rectangle is constructed by the Image's mObjectFactory
        .
34     void AddRectangle(size_t const x, size_t const y, std::string
        const& color, size_t const width, size_t const height);
35
36
37     /// @brief Adds a new Circle to the currently selected Image.
38     /// @param[in] x The X-coordinate of position.
39     /// @param[in] y The Y-coordinate of position.
40     /// @param[in] color Color of the circle (e.g. black, green
        ,...).
41     /// @param[in] radius The radius of the circle.
42     ///
43     /// The Circle is constructed by the Image's mObjectFactory.
44     void AddCircle(size_t const x, size_t const y, std::string
        const& color, size_t const radius);
45
46     /// @brief Writes all Image objects to a file in SVG-Format.
47     ///
48     /// The name of each created file is <Image.mName>.svg.
49     void WriteAllSVG() const;
50
51 private:
52     /// @brief Default CTor.
```

```
53      ImageManager() = default;
54
55      /// @brief Stores multiple images.
56      std::vector<Image::UPtr> mImages;
57  };
58
59  #endif
```

Listing 26: Beispiel01/ImageManager.cpp

```
1   #include "ImageManager.h"
2
3   #include <stdexcept>
4
5   static auto const cErrNoImg = "No image selected";
6
7   void ImageManager::AddImage(std::string const& name,
        GraphicObjectFactory& factory)
8   {
9       mImages.emplace_back(std::make_unique<Image>(name, factory));
10  }
11
12  void ImageManager::WriteAllSVG() const
13  {
14      for (auto const& img : mImages)
15      {
16          if (img != nullptr) img->WriteToSVG();
17      }
18  }
19
20  void ImageManager::AddRectangle(size_t const x, size_t const y,
        std::string const& color, size_t const width, size_t const
        height)
21  {
22      if (mImages.empty()) throw std::runtime_error(cErrNoImg);
23
24      mImages.back()->AddRectangle(x, y, color, width, height);
25  }
26
27
28  void ImageManager::AddCircle(size_t const x, size_t const y, std::
        string const& color, size_t const radius)
29  {
30      if (mImages.empty()) throw std::runtime_error(cErrNoImg);
31
32      mImages.back()->AddCircle(x, y, color, radius);
33  }
```

6.7 Testtreiber

Listing 27: "Beispiel01/Test.cpp"

```
1  #include "gtest/gtest.h"
2
3  #include <vld.h>
4  #include <fstream>
5  #include <sstream>
6  #include <filesystem>
7
8  #include "FramedRectangle.h"
9  #include "FilledRectangle.h"
10 #include "FramedCircle.h"
11 #include "FilledCircle.h"
12 #include "FramedObjectFactory.h"
13 #include "FilledObjectFactory.h"
14 #include "Image.h"
15 #include "ImageManager.h"
16
17 using namespace std;
18
19 ////////////////////////////////////////////////
20
21 TEST(FramedRectangle, Test) {
22     FramedRectangle const rect1{ 0, 0, "black", 10, 20 };
23     FramedRectangle const rect2{ 100, 100,  "doesnotexist", 0, 0
           };
24     EXPECT_EQ(rect1.ToSVG(), "<rect x=\"0\" y=\"0\" width=\"10\"
           height=\"20\" stroke=\"black\" fill=\"none\"/>");
25     EXPECT_EQ(rect2.ToSVG(), "<rect x=\"100\" y=\"100\" width
           =\"0\" height=\"0\" stroke=\"doesnotexist\" fill=\"none
           \"/>");
26
27     FramedRectangle const rect3{ rect1 };
28
29     EXPECT_EQ(rect3.ToSVG(), rect1.ToSVG());
30 }
31
32 ////////////////////////////////////////////////
33
34 TEST(FilledRectangle, Test) {
35     FilledRectangle const rect1{ 0, 0, "black", 10, 20 };
36     FilledRectangle const rect2{ 100, 100,  "doesnotexist", 0, 0
           };
37     EXPECT_EQ(rect1.ToSVG(), "<rect x=\"0\" y=\"0\" width=\"10\"
           height=\"20\" stroke=\"black\" fill=\"black\"/>");
38     EXPECT_EQ(rect2.ToSVG(), "<rect x=\"100\" y=\"100\" width
           =\"0\" height=\"0\" stroke=\"doesnotexist\" fill=\"
           doesnotexist\"/>");
39
40     FilledRectangle const rect3{ rect1 };
41
42     EXPECT_EQ(rect3.ToSVG(), rect1.ToSVG());
43 }
44
45 ////////////////////////////////////////////////
46
```

```
47 TEST(FramedCircle, Test) {
48     FramedCircle const circ1{ 0, 0, "black", 10};
49     FramedCircle const circ2{ 100, 100,  "doesnotexist", 0};
50     EXPECT_EQ(circ1.ToSVG(), "<circle cx=\"0\" cy=\"0\" r=\"10\"
           stroke=\"black\" fill=\"none\"/>");
51     EXPECT_EQ(circ2.ToSVG(), "<circle cx=\"100\" cy=\"100\" r
           =\"0\" stroke=\"doesnotexist\" fill=\"none\"/>");
52
53     FramedCircle const circ3{ circ1 };
54
55     EXPECT_EQ(circ3.ToSVG(), circ1.ToSVG());
56 }
57
58 ////////////////////////////////////////////////
59
60 TEST(FilledCircle, Test) {
61     FilledCircle const circ1{ 0, 0, "black", 10 };
62     FilledCircle const circ2{ 100, 100,  "doesnotexist", 0 };
63     EXPECT_EQ(circ1.ToSVG(), "<circle cx=\"0\" cy=\"0\" r=\"10\"
           stroke=\"black\" fill=\"black\"/>");
64     EXPECT_EQ(circ2.ToSVG(), "<circle cx=\"100\" cy=\"100\" r
           =\"0\" stroke=\"doesnotexist\" fill=\"doesnotexist\"/>");
65
66     FilledCircle const circ3{ circ1 };
67
68     EXPECT_EQ(circ3.ToSVG(), circ1.ToSVG());
69 }
70
71 ////////////////////////////////////////////////
72
73 TEST(FramedObjectFactory, Test) {
74     FramedObjectFactory& fact = FramedObjectFactory::GetInstance()
           ;
75
76     GraphicObject::UPtr circ1 = fact.CreateCircle(10, 10, "black",
            3);
77     FramedCircle const circ1Manual = FramedCircle{ 10, 10, "black"
           , 3 };
78     EXPECT_EQ(circ1->ToSVG(), circ1Manual.ToSVG());
79
80     GraphicObject::UPtr rect1 = fact.CreateRectangle(10, 10, "
           black", 3, 5);
81     FramedRectangle const rect1Manual = FramedRectangle{ 10, 10, "
           black", 3, 5 };
82     EXPECT_EQ(rect1->ToSVG(), rect1Manual.ToSVG());
83 }
84
85 ////////////////////////////////////////////////
86
87 TEST(FilledObjectFactory, Test) {
88     FilledObjectFactory& fact = FilledObjectFactory::GetInstance()
           ;
89
90     GraphicObject::UPtr circ1 = fact.CreateCircle(10, 10, "black",
            3);
```

```
91      FilledCircle const circ1Manual = FilledCircle{ 10, 10, "black"
            , 3 };
92      EXPECT_EQ(circ1->ToSVG(), circ1Manual.ToSVG());
93
94      GraphicObject::UPtr rect1 = fact.CreateRectangle(10, 10, "
            black", 3, 5);
95      FilledRectangle const rect1Manual = FilledRectangle{ 10, 10, "
            black", 3, 5 };
96      EXPECT_EQ(rect1->ToSVG(), rect1Manual.ToSVG());
97  }
98
99  ////////////////////////////////////////////////
100
101 TEST(Image, Test) {
102     filesystem::remove("TestImage.svg");
103     ifstream file;
104     ostringstream stream;
105     Image img{ "TestImage", FilledObjectFactory::GetInstance() };
106
107     img.WriteToSVG();
108
109     file.open("TestImage.svg");
110     EXPECT_EQ(file.is_open(), true);
111     stream << file.rdbuf();
112     EXPECT_EQ(stream.str(), "<?xml version=\"1.0\"?>\n<svg xmlns
            =\"http://www.w3.org/2000/svg\">\n</svg>");
113     file.close();
114     stream.str("");
115
116     img.AddCircle(20, 20, "green", 10);
117     img.AddCircle(30, 30, "red", 20);
118     img.AddRectangle(0, 0, "black", 10, 20);
119
120     img.WriteToSVG();
121     file.open("TestImage.svg");
122     EXPECT_EQ(file.is_open(), true);
123     stream << file.rdbuf();
124     EXPECT_EQ(stream.str(), "<?xml version=\"1.0\"?>\n<svg xmlns
            =\"http://www.w3.org/2000/svg\">\n<circle cx=\"20\" cy
            =\"20\" r=\"10\" stroke=\"green\" fill=\"green\"/>\n<
            circle cx=\"30\" cy=\"30\" r=\"20\" stroke=\"red\" fill=\"
            red\"/>\n<rect x=\"0\" y=\"0\" width=\"10\" height=\"20\"
            stroke=\"black\" fill=\"black\"/>\n</svg>");
125     file.close();
126     stream.str("");
127 }
128
129 TEST(Image, InvldName) {
130     try {
131         Image img{ "", FilledObjectFactory::GetInstance() };
132         EXPECT_EQ(1, 0); // Fail.
133     }
134     catch (std::invalid_argument const& ex) {
135         EXPECT_STREQ(ex.what(), "Invalid name");
136     }
```

```
137
138     Image const img{ "doesnotexist/file", FilledObjectFactory::
            GetInstance() };
139     try {
140         img.WriteToSVG();
141         EXPECT_EQ(1, 0); // Fail.
142     }
143     catch (std::runtime_error const& ex) {
144         EXPECT_STREQ(ex.what(), "Error opening file");
145     }
146 }
147
148 ////////////////////////////////////////////////
149
150 TEST(ImageManager, Test) {
151     filesystem::remove("Img1.svg");
152     filesystem::remove("Img2.svg");
153     filesystem::remove("Img3.svg");
154     ifstream file;
155     ostringstream stream;
156
157     ImageManager& mgr = ImageManager::GetInstance();
158     mgr.WriteAllSVG();
159     EXPECT_EQ(filesystem::exists("Img1.svg"), false);
160     EXPECT_EQ(filesystem::exists("Img2.svg"), false);
161     EXPECT_EQ(filesystem::exists("Img3.svg"), false);
162
163     try {
164         mgr.AddCircle(100, 100, "yellow", 10);
165         EXPECT_EQ(1, 0); // Fail.
166     }
167     catch (std::runtime_error const& ex) {
168         EXPECT_STREQ(ex.what(), "No image selected");
169     }
170     try {
171         mgr.AddRectangle(100, 100, "yellow", 100, 200);
172         EXPECT_EQ(1, 0); // Fail.
173     }
174     catch (std::runtime_error const& ex) {
175         EXPECT_STREQ(ex.what(), "No image selected");
176     }
177
178     mgr.AddImage("Img1", FramedObjectFactory::GetInstance());
179     mgr.WriteAllSVG();
180     EXPECT_EQ(filesystem::exists("Img1.svg"), true);
181     EXPECT_EQ(filesystem::exists("Img2.svg"), false);
182     EXPECT_EQ(filesystem::exists("Img3.svg"), false);
183
184     file.open("Img1.svg");
185     EXPECT_EQ(file.is_open(), true);
186     stream << file.rdbuf();
187     EXPECT_EQ(stream.str(), "<?xml version=\"1.0\"?>\n<svg xmlns
            =\"http://www.w3.org/2000/svg\">\n</svg>");
188     file.close();
189     stream.str("");
```

```
190
191     mgr.AddCircle(100, 100, "blue", 50);
192     mgr.AddRectangle(50, 50, "red", 200, 300);
193     mgr.WriteAllSVG();
194     EXPECT_EQ(filesystem::exists("Img1.svg"), true);
195     EXPECT_EQ(filesystem::exists("Img2.svg"), false);
196     EXPECT_EQ(filesystem::exists("Img3.svg"), false);
197
198     file.open("Img1.svg");
199     EXPECT_EQ(file.is_open(), true);
200     stream << file.rdbuf();
201     EXPECT_EQ(stream.str(), "<?xml version=\"1.0\"?>\n<svg xmlns
            =\"http://www.w3.org/2000/svg\">\n<circle cx=\"100\" cy
            =\"100\" r=\"50\" stroke=\"blue\" fill=\"none\"/>\n<rect x
            =\"50\" y=\"50\" width=\"200\" height=\"300\" stroke=\"red
            \" fill=\"none\"/>\n</svg>");
202     file.close();
203     stream.str("");
204
205     mgr.AddImage("Img2", FilledObjectFactory::GetInstance());
206     mgr.AddImage("Img3", FilledObjectFactory::GetInstance());
207     mgr.AddCircle(200, 200, "black", 100);
208     mgr.AddCircle(200, 200, "red", 90);
209     mgr.AddRectangle(300, 300, "yellow", 100, 100);
210
211     mgr.WriteAllSVG();
212     EXPECT_EQ(filesystem::exists("Img1.svg"), true);
213     EXPECT_EQ(filesystem::exists("Img2.svg"), true);
214     EXPECT_EQ(filesystem::exists("Img3.svg"), true);
215
216     file.open("Img1.svg");
217     EXPECT_EQ(file.is_open(), true);
218     stream << file.rdbuf();
219     EXPECT_EQ(stream.str(), "<?xml version=\"1.0\"?>\n<svg xmlns
            =\"http://www.w3.org/2000/svg\">\n<circle cx=\"100\" cy
            =\"100\" r=\"50\" stroke=\"blue\" fill=\"none\"/>\n<rect x
            =\"50\" y=\"50\" width=\"200\" height=\"300\" stroke=\"red
            \" fill=\"none\"/>\n</svg>");
220     file.close();
221     stream.str("");
222
223     file.open("Img2.svg");
224     EXPECT_EQ(file.is_open(), true);
225     stream << file.rdbuf();
226     EXPECT_EQ(stream.str(), "<?xml version=\"1.0\"?>\n<svg xmlns
            =\"http://www.w3.org/2000/svg\">\n</svg>");
227     file.close();
228     stream.str("");
229
230     file.open("Img3.svg");
231     EXPECT_EQ(file.is_open(), true);
232     stream << file.rdbuf();
233     EXPECT_EQ(stream.str(), "<?xml version=\"1.0\"?>\n<svg xmlns
            =\"http://www.w3.org/2000/svg\">\n<circle cx=\"200\" cy
            =\"200\" r=\"100\" stroke=\"black\" fill=\"black\"/>\n<
```

```
        circle cx=\"200\" cy=\"200\" r=\"90\" stroke=\"red\" fill
        =\"red\"/>\n<rect x=\"300\" y=\"300\" width=\"100\" height
        =\"100\" stroke=\"yellow\" fill=\"yellow\"/>\n</svg>");
234     file.close();
235     stream.str("");
236 }
237
238 TEST(ImageManager, InvldName) {
239     try {
240         ImageManager::GetInstance().AddImage("",
                FilledObjectFactory::GetInstance());
241         EXPECT_EQ(0, 1); // Fail.
242     }
243     catch (std::invalid_argument const& ex) {
244         EXPECT_STREQ(ex.what(), "Invalid name");
245     }
246
247     ImageManager::GetInstance().AddImage("doesnotexist/file",
            FilledObjectFactory::GetInstance());
248     try {
249         ImageManager::GetInstance().WriteAllSVG();
250         EXPECT_EQ(1, 0); // Fail.
251     }
252     catch (std::runtime_error const& ex) {
253         EXPECT_STREQ(ex.what(), "Error opening file");
254     }
255 }
```